

14B. Reasoning Models — Os Modelos Que Pensam Antes de Falar

“Quando o modelo aprende a rascunhar antes de responder, a fatura cresce em tokens invisíveis, mas o erro em decisões caras tende a cair de forma desproporcional, e o desafio do operador maduro é decidir onde essa troca compensa.”

14B.1 — O CONCEITO INTUITIVO

Existe uma diferença sutil mas estruturalmente importante entre instruir um modelo de propósito geral a “pensar passo a passo” antes de responder, técnica que vimos no Capítulo 10 sob o nome de Chain of Thought, e usar um modelo cuja arquitetura ou cujo treinamento foi otimizado para raciocinar em profundidade como parte essencial do processo de inferência. A primeira é uma instrução de prompt, frágil porque depende da disposição do modelo em obedecê-la, limitada porque a cadeia gerada compete por espaço de contexto com a resposta final, e visível porque cada token de raciocínio aparece para o usuário. A segunda é um produto, em que o modelo foi treinado especificamente para reservar uma fase de pensamento prolongada antes de comprometer-se com a resposta, e onde essa fase pode consumir dezenas ou centenas de milhares de tokens internos sem que o usuário veja diretamente o conteúdo, embora pague por todos eles na conta.

Essa classe de modelos, que ficou conhecida no mercado como reasoning models, ou modelos de raciocínio, ou em algumas nomenclaturas modelos com pensamento estendido, emergiu como categoria comercial entre o segundo semestre de 2024 e o ano de 2025, com lançamentos da OpenAI na família o, da Anthropic com modos de raciocínio estendido em Claude, da Google com versões de Gemini orientadas a raciocínio profundo, e principalmente com a publicação do DeepSeek R1 em janeiro de 2025, cujo paper na Nature em 2025 forneceu o primeiro blueprint público completo de como construir um reasoning model partindo de aprendizado por reforço puro, sem fase supervisionada inicial, e como o comportamento de raciocínio emergiu de forma surpreendente durante o treinamento. Aquele paper é o ponto de virada que vou usar como referência principal neste capítulo, porque é tanto o documento técnico mais detalhado disponível publicamente sobre como reasoning models são construídos, quanto o trabalho que abriu o caminho para que pesquisadores em todo lugar pudessem replicar e iterar sobre a ideia, mudando a tração da pesquisa em raciocínio de algo restrito a poucos laboratórios bem financiados para algo amplamente acessível.

O ponto importante que vale fixar desde já, antes de entrar em detalhe técnico, é que reasoning model não é apenas Chain of Thought embrulhado em invólucro comercial, ainda que muitos descrevam dessa forma de maneira simplificada e enganosa. Existe diferença mensurável de qualidade, em tarefas específicas, entre instruir um modelo de propósito geral a pensar passo a passo, e usar um modelo treinado para pensar de forma muito mais extensa e estruturada. Mas existe também diferença mensurável de custo e de latência, em ordens de magnitude que tornam a decisão de qual usar não trivial, e que merecem ser tratadas com a mesma seriedade com que se trata qualquer decisão arquitetural em sistemas de produção. O leitor que sai deste capítulo apenas com a impressão de que reasoning models são uma versão melhor de tudo e deveriam ser usados sempre que possível terá entendido pela metade, e a metade não entendida é justamente onde o operador sênior se distingue do entusiasta.

Cabe ainda introduzir uma terceira camada, mais filosófica mas com consequências práticas que vou explorar adiante, que é a questão da faithfulness do raciocínio gerado por esses modelos, ou seja, em que medida a cadeia de pensamento que o modelo articula reflete o cálculo interno que efetivamente o levou à conclusão final. A pesquisa publicada por Anthropic e por outros laboratórios entre 2023 e 2025, com nomes como Lanham e colegas conduzindo experimentos sistemáticos, sugere que essa correspondência não é total nem garantida, e que o raciocínio exposto pode ser, em casos identi-

ficáveis, um relato pós-hoc plausível e não o cálculo real. Essa descoberta não invalida reasoning models, mas adiciona um nível de cautela necessário em qualquer aplicação em que o raciocínio articulado vá ser usado como evidência de qualidade, como aconteceria em domínios regulados, jurídicos, médicos, ou em qualquer situação em que auditabilidade do raciocínio seja parte do produto.

14B.2 — ANALOGIA: O ENGENHEIRO QUE RASCUNHA NO PAPEL ANTES DE COMITAR

Para tornar tangível a diferença entre um modelo de propósito geral instruído a raciocinar e um reasoning model, vale considerar uma analogia do dia a dia da engenharia de software, que cobre boa parte do leitor desta obra e produz intuição relativamente precisa. Imagine dois engenheiros, ambos seniores, ambos competentes, trabalhando numa mesma equipe sob a mesma pressão de prazo. O primeiro recebe uma tarefa complexa de arquitetura, abre o editor e começa imediatamente a escrever código, refinando à medida que vai progredindo, com pequenas pausas para revisão mental mas sem nenhum artefato externo registrando o raciocínio intermediário. Entrega a feature funcional em três dias e segue para a próxima tarefa.

O segundo engenheiro, recebendo a mesma tarefa, dedica o primeiro dia a rascunhar no papel, ou em um documento de design, todas as alternativas que considerou, os trade-offs entre cada uma, os casos de borda que precisam ser tratados, as suposições que está fazendo sobre o resto do sistema, as decisões que vai precisar revisar conforme o trabalho avance. Só no segundo dia abre o editor, e nos dias seguintes implementa com base nesse rascunho, refinando o documento à medida que descobre coisas durante a implementação. Entrega a feature funcional em quatro dias, um dia a mais que o primeiro, mas com documento de design pronto, com decisões arquiteturais explícitas, com cobertura mais ampla de casos de borda, e com um artefato de revisão que sobrevive à entrega e ajuda quem vai manter o código depois.

Os dois engenheiros podem ter entregado funcionalidade equivalente, mas o segundo trabalhou em modo qualitativamente diferente. Ele pagou um dia a mais, ou seja, vinte e cinco por cento de custo adicional em tempo, em troca de um processo mais robusto, com menos chance de surpresa em produção e com auditabilidade do ra-

ciocínio. Para tarefas triviais, o primeiro engenheiro é a escolha óbvia, porque o overhead do rascunho não compensa o ganho marginal de qualidade num código que já seria correto sem ele. Para tarefas críticas, com consequência operacional séria, regulatória ou financeira, o segundo engenheiro é não apenas a escolha óbvia, mas a única escolha defensável, e a pressa do primeiro vira fonte de dívida técnica que cobra juros em incidentes futuros.

Reasoning models são o equivalente computacional do segundo engenheiro, e modelos de propósito geral em modo zero-shot ou com Chain of Thought instruído via prompt são variações do primeiro. A diferença não está apenas em quanto pensam, mas em como pensam, em quanto isso custa, em quanto demora, e em quão útil é para o tipo de decisão que está sendo tomada. Trazer essa analogia para a conversa com o time é frequentemente o caminho mais curto para que executivos não-técnicos entendam por que reasoning model não é simplesmente “o modelo melhor”, e por que escolher um sem entender quando ele compensa é forma cara de errar.

14B.3 — EXPLICAÇÃO TÉCNICA

14B.3.1 — RL puro como caminho de raciocínio emergente: o blueprint DeepSeek R1

Para entender por que reasoning models funcionam, e por que o trabalho do DeepSeek em janeiro de 2025 mudou a conversa em torno desses modelos, vale percorrer brevemente o caminho técnico que aquele paper documenta, sem entrar em código nem em números específicos de benchmark, porque ambos pertencem ao Apêndice J e porque o que importa aqui é o princípio estrutural. A receita tradicional de treinar um modelo de linguagem para tarefas complexas envolvia uma fase inicial de pré-treinamento sobre texto da internet, seguida de uma fase de fine-tuning supervisionado em que humanos rotulam exemplos de cadeia de raciocínio desejada, seguida de uma fase de aprendizado por reforço com feedback humano, ou RLHF, que ajusta o modelo para preferências humanas. Essa receita produziu os modelos de propósito geral que conhecemos, e funciona razoavelmente bem para uma ampla gama de tarefas.

O que o time do DeepSeek demonstrou, e publicou de forma detalhada na Nature em 2025, é que essa segunda fase de fine-tuning supervisionado pode ser pulada quando o objetivo é raciocínio em domínios verificáveis, como matemática e código, e que aplicando

aprendizado por reforço diretamente sobre o modelo pré-treinado, com função de recompensa baseada apenas em verificar se a resposta final está correta, o modelo desenvolve sozinho, ao longo do treinamento, cadeias de raciocínio cada vez mais longas, com revisões, com auto-correções, com expressões espontâneas equivalentes a “espera, deixa eu reconsiderar isso”, e com comportamento que se assemelha cada vez mais ao de um humano deliberando sobre um problema difícil. O ponto que vale grifar é que esse comportamento emergiu, ou seja, ninguém ensinou diretamente o modelo a pensar dessa forma. O sinal de recompensa foi apenas “a resposta está certa ou errada”, e o modelo descobriu, durante o treinamento, que pensar mais antes de responder aumenta a chance de acertar, e que estruturas de raciocínio cada vez mais sofisticadas levam a acertos mais consistentes.

Esse achado é estruturalmente importante por duas razões que vale articular. A primeira é metodológica, porque mostra que uma das fases mais caras e mais dependentes de trabalho humano qualificado no treinamento de modelos de raciocínio, ou seja, o fine-tuning supervisionado com exemplos rotulados de raciocínio, pode ser eliminada ou drasticamente reduzida quando existe verificador automático da resposta final, o que abre o caminho para muitos laboratórios menores produzirem reasoning models competentes sem o custo proibitivo daquela fase. A segunda é conceitual, porque sugere que o comportamento de raciocínio prolongado não é uma habilidade externa que precisa ser ensinada explicitamente, mas algo que emerge naturalmente quando o modelo é incentivado a maximizar acerto em problemas onde acertar é difícil, o que tem implicações filosóficas e práticas sobre como pensar a respeito do que esses modelos estão fazendo.

Vale uma ressalva de honestidade aqui, em linha com o Invariante 1 de Plausibilidade que governa este capítulo. Mesmo nos reasoning models mais sofisticados, treinados com aprendizado por reforço sobre verificadores robustos, o modelo continua otimizando saída plausível, não saída verdadeira em sentido pleno. Em domínios onde existe verificador objetivo da resposta final, como matemática e código, a saída plausível tende a coincidir com a saída verdadeira porque o treinamento penaliza divergência. Em domínios onde o verificador é fraco ou inexistente, como análise estratégica, parecer jurídico discursivo, ou avaliação de risco em situação inédita, a cadeia de raciocínio prolongada do reasoning model continua produzindo saída plausível, e mais cadeia significa mais oportunidade de diver-

gir da realidade de forma sofisticada e difícil de detectar. Esse ponto será aprofundado na seção 14B.5 sobre quando reasoning ganha de zero-shot CoT e quando perde.

14B.3.2 — Diferença entre zero-shot CoT, few-shot CoT e reasoning model treinado

REASONING MODELS — TRÊS MODOS, USOS DISTINTOS		
Não é "modelo melhor" - é categoria com perfil próprio de custo, latência e auditabilidade		
ZERO-SHOT CoT "Pense passo a passo" Custo 2-5x tokens de resposta direta Latência Proporcional aos tokens Qualidade Boa em complexidade moderada Auditabilidade 100% - cadeia totalmente visível Quando usar Tarefa de complexidade média Orçamento de tokens apertado Auditoria completa exigida Exemplos Claude 4.7 · GPT-5.5 · Gemini 3.1 (modo padrão · prompt CoT)	FEW-SHOT CoT CoT por imitação de exemplos Custo Alto em input (exemplos pesam) Latência Maior que zero-shot Qualidade Calibrada para o domínio Auditabilidade 100% - cadeia totalmente visível Quando usar Estrutura de raciocínio específica Análise de contrato, parecer Diagnóstico clínico estruturado Exemplos Qualquer modelo de propósito geral com 3-5 exemplos em prompt	REASONING MODEL Pensamento treinado por RL Custo 5-20x CoT · tier separado Latência Dezenas de segundos Qualidade Superior em mat./código/planejamento Auditabilidade Parcial - cadeia oculta ou resumida Quando usar Matemática avançada, código complexo Planejamento multipasso Tolerância de latência alta Exemplos o3, o4-mini · Claude Adaptive · DeepSeek R1 · Gemini extended
Roteamento maduro escolhe o modo por tarefa, não trata reasoning como "modelo melhor".		

Três modos de raciocínio comparados — custo, latência, qualidade e auditabilidade

Existem três modos diferentes de fazer um modelo articular raciocínio antes de produzir resposta, e o operador maduro precisa saber distinguir os três com precisão, porque cada um tem perfil de custo, latência, qualidade e auditabilidade que justifica usos diferentes. O Capítulo 10 cobriu Chain of Thought em detalhe, e este capítulo deve ser lido em sequência com aquele para que o leitor consiga construir o mapa completo, mas vou recolocar a distinção aqui em termos comparativos para fixar o que reasoning model adiciona.

No modo zero-shot Chain of Thought, o operador adiciona ao prompt uma instrução genérica como “pense passo a passo antes de responder”, e o modelo de propósito geral, treinado com RLHF que reforça esse padrão, produz uma cadeia visível de raciocínio antes da resposta final. O custo adicional é da ordem de dois a cinco vezes os tokens da resposta direta, a latência adicional é proporcional, e a cadeia gerada fica integralmente visível para o usuário e auditável

de imediato. Funciona surpreendentemente bem em problemas de complexidade moderada, e é frequentemente subutilizada em organizações que ainda não dominaram engenharia de prompt.

No modo few-shot Chain of Thought, o operador inclui no prompt exemplos completos de problemas resolvidos com raciocínio explícito no formato desejado, ensinando o modelo por imitação a estruturar o raciocínio de forma específica para aquele domínio. É mais caro em tokens de entrada porque os exemplos pesam, é mais preciso porque calibra o tipo de raciocínio, e mantém a mesma visibilidade total da cadeia gerada. Vale especialmente em domínios técnicos onde a estrutura do raciocínio desejado é específica, como análise de contrato, parecer técnico-financeiro, diagnóstico clínico.

No modo reasoning model treinado, o modelo foi otimizado por aprendizado por reforço para reservar uma fase de pensamento extensa antes da resposta, e essa fase pode consumir uma ordem de magnitude mais tokens que qualquer Chain of Thought instruído por prompt, com a diferença adicional de que parte ou totalidade desses tokens pode ficar oculta para o usuário, dependendo da política do provedor. A latência cresce proporcionalmente, podendo passar de dezenas de segundos para problemas complexos. A qualidade em problemas onde raciocínio profundo importa é mensuravelmente superior, especialmente em matemática avançada, código complexo, planejamento multipasso. E a auditabilidade do raciocínio é parcial, porque o usuário vê o resumo ou o resultado da fase de pensamento, mas não necessariamente todo o conteúdo gerado internamente, o que cria implicações de governança que tratarei em 14B.3.3.

A consequência operacional dessa distinção é que reasoning model não é o substituto natural de Chain of Thought, e sim uma categoria adicional na caixa de ferramentas, com casos de uso próprios. Operações maduras em 2026 já têm rotinas explícitas de roteamento entre os três modos, escolhendo conforme a complexidade da tarefa, a tolerância de latência, o custo por chamada admissível, e a necessidade de auditabilidade completa. Equipes que tratam reasoning model como “modelo melhor” e o aplicam de forma indiscriminada normalmente descobrem, em algumas semanas, que a fatura de tokens cresceu de forma que ninguém consegue defender em comitê de orçamento.

14B.3.3 — O pensamento como cadeia oculta versus exposta: a questão da faithfulness

A forma como os provedores comerciais lidam com a cadeia de raciocínio gerada por reasoning models varia, e essa variação tem consequência operacional que merece atenção. Alguns provedores expõem integralmente a cadeia, deixando o usuário ler cada token de pensamento. Outros expõem apenas um resumo curado do raciocínio, em parte para proteger propriedade intelectual sobre o conteúdo das cadeias, em parte porque cadeias longas frequentemente contêm material desorganizado que não agrega para o usuário final. Outros ocultam totalmente a cadeia, expondo apenas a resposta final. Cada decisão de exposição tem implicações sobre auditabilidade, sobre custo de armazenamento de logs, e sobre como o sistema pode ser usado em domínios regulados.

Sobre essa exposição da cadeia, vale articular um ponto que vem ganhando peso na literatura técnica recente, e que é central para qualquer aplicação que pretenda usar reasoning models em domínios sensíveis. A questão da faithfulness, ou fidelidade do raciocínio, refere-se a em que medida a cadeia de pensamento que o modelo articula reflete efetivamente o cálculo interno que o levou à conclusão. Trabalhos publicados por Anthropic, por Lanham e colegas, e por outros grupos de pesquisa entre 2023 e 2025, conduziram experimentos sistemáticos para testar essa correspondência, em que se altera o prompt de formas que deveriam mudar o raciocínio, e se observa se a cadeia exposta acompanha essa mudança ou se permanece relativamente estável enquanto a resposta final muda por outras razões.

Os resultados desses experimentos são desconfortáveis para quem trata reasoning models como caixas transparentes. Em casos identificáveis, especialmente sob certas condições de prompt, a cadeia de raciocínio exposta pode parecer racionalizar uma conclusão à qual o modelo chegou por outros caminhos que não estão refletidos no texto da cadeia. Isso não é mentira intencional do modelo, é consequência da forma como esses sistemas operam, com a cadeia gerada compartilhando arquitetura com a resposta final, e ambos sendo otimizados conjuntamente para parecer coerentes. O risco operacional é que organizações em domínios regulados podem estar tratando o raciocínio articulado como evidência de processo decisório auditável, quando o que estão tendo é uma narrativa plausível que acompanha a conclusão.

Esse ponto não invalida o uso de reasoning models. Em muitos domínios, e especialmente em matemática e código onde existe verificador objetivo da resposta final, a cadeia de raciocínio tende a ser razoavelmente fiel ao cálculo interno porque o treinamento penaliza divergência. Em outros domínios, especialmente análises discursivas e decisões estratégicas, a fidelidade do raciocínio articulado precisa ser tratada com ceticismo, e o operador maduro complementa o uso de reasoning models com mecanismos externos de auditoria do raciocínio, como avaliação humana de amostras representativas, evals específicos para detectar racionalização pós-hoc, e protocolos de redundância em decisões com alto custo de erro.

14B.3.4 — Custo de inferência: tokens de pensamento entram na fatura

□ **Nota editorial sobre não-redundância.** Esta subseção trata custo composto sob a lente específica do raciocínio prolongado, com foco em por que o token oculto entra na fatura e em como instrumentar o controle quando reasoning model entra em produção. O tratamento canônico do tema vive no Capítulo 18 — Economia de Tokens e no Framework F7 — Custo Composto, com as três alavancas (chamadas $\times$ redundância $\times$ tier) e as cinco frentes de otimização. A leitura cruzada economiza relectura e mantém cada capítulo no seu papel: este capítulo é sobre quando ativar reasoning; C18 e F7 são sobre como pagar a fatura sem comprometer a operação.

Existe um aspecto comercial dos reasoning models que precisa ser tratado de frente, porque é a fonte mais comum de surpresa operacional para equipes que migram de modelos de propósito geral para reasoning models sem instrumentação adequada. Os tokens gerados durante a fase de pensamento são tokens de output do ponto de vista da fatura, e são cobrados como tal, mesmo quando não são expostos ao usuário. Em problemas suficientemente complexos para justificar reasoning model, essa fase pode gerar entre cinco e vinte vezes mais tokens do que a resposta visível, o que multiplica diretamente o custo por chamada na mesma proporção.

Essa multiplicação compõe com outros eixos de custo descritos no Capítulo 18 sobre economia de tokens e no Framework F7 sobre Custo Composto. O tier de reasoning é tipicamente cobrado em escala separada dos tiers padrão do mesmo provedor, com preço unitário por token frequentemente maior, justamente porque o prove-

dor sabe que o caso de uso justifica disposição a pagar mais por chamada quando o problema importa. A combinação de mais tokens por chamada e preço unitário maior produz fatura por chamada que pode estar uma ordem de magnitude acima do que a mesma equipe pagava em modelo de propósito geral para tarefas equivalentes.

A operação madura responde a essa realidade com três disciplinas. A primeira é instrumentação granular do custo, segmentado por funcionalidade e por usuário, com dashboards que mostram em tempo real onde o gasto de reasoning está concentrado. A segunda é roteamento inteligente, em que tarefas que não requerem reasoning são desviadas para modelos de propósito geral via lógica explícita, e apenas as tarefas que efetivamente se beneficiam de raciocínio profundo chegam ao reasoning model. A terceira é controle de orçamento por chamada e por dia, com cortes automatizados se algo dispare, e com alertas para investigação de anomalia antes que vire incidente financeiro. Equipes que não implementam essas três disciplinas e mesmo assim adotam reasoning models normalmente recebem a fatura no fim do mês e descobrem, à força, por que o Custo Composto (Princípio 5) é um dos princípios estruturais desta obra.

Vale ainda registrar, sem entrar em números porque versões e preços estão no Apêndice J e mudam constantemente, que a tendência observada entre 2024 e 2026 foi de queda relativa do custo unitário por token de reasoning à medida que provedores otimizam suas operações e a competição se intensifica. Mas essa queda relativa é frequentemente compensada, e em alguns casos superada, pela tendência paralela de cadeias de raciocínio cada vez mais longas conforme os modelos melhoram, de forma que o custo por chamada útil tem se mantido alto mesmo com tokens individuais ficando mais baratos. Quem planeja com base na ideia de que o custo cairá significativamente nos próximos anos pode estar otimista demais.

14B.3.5 — Quando reasoning ganha de zero-shot CoT e quando perde

Para fechar a discussão técnica, vale articular com precisão em que tipos de tarefa reasoning model entrega ganho real sobre Chain of Thought instruído via prompt em modelo de propósito geral, e em que tipos a vantagem não se materializa ou até se inverte. Esse mapeamento é o que separa decisão informada de modismo, e vale construir mentalmente como heurística antes de adotar reasoning model como padrão em qualquer fluxo.

Reasoning model ganha de forma clara em tarefas com cadeia longa de dedução, em que cada passo depende rigorosamente do anterior e um erro no meio compromete a resposta final. Isso inclui matemática avançada, demonstrações formais, otimização combinatória, planejamento multipasso com restrições, código complexo que exige análise de múltiplas interações entre componentes. Em todos esses casos, a fase de pensamento prolongada permite ao modelo explorar alternativas, recuar quando detecta inconsistência, e chegar à resposta final com qualidade que Chain of Thought em modelo de propósito geral raramente alcança.

Reasoning model perde de forma clara em tarefas rotineiras com baixo custo de erro, em que a resposta direta de um modelo de propósito geral já seria adequada e o overhead de raciocínio profundo apenas adiciona latência e custo sem ganho mensurável. Classificação simples, extração de campo, tradução curta, resposta a pergunta factual estabelecida, são exemplos onde reasoning model é desperdício puro. Perde também em fluxos conversacionais, em que a latência da fase de pensamento quebra a experiência do usuário e onde a tolerância a tempo de resposta acima de poucos segundos é inexistente em produção.

Existem ainda casos intermediários, onde a decisão não é óbvia e exige experimentação informada. Tarefas de complexidade moderada, em que CoT em modelo de propósito geral resolve a maior parte dos casos mas falha em uma fração identificável, podem se beneficiar de roteamento dinâmico, em que apenas os casos com sinais de complexidade alta são desviados para reasoning model, e os demais seguem em modelo padrão. Esse padrão de cascata, em que tarefa fácil resolve no modelo barato e apenas a fração difícil ativa o modelo caro, é frequentemente a arquitetura economicamente defensável em operações com volume relevante.

Existe ainda uma categoria onde reasoning model surpreendentemente perde, e que vale conhecer porque é contraintuitiva. Em tarefas onde a resposta certa depende fortemente de conhecimento factual específico, e não tanto de raciocínio sobre conhecimento, modelos de propósito geral grandes frequentemente entregam respostas equivalentes ou superiores, porque o reasoning model gastou ciclos pensando sobre algo que o conhecimento direto já resolveria. Sintomas como “qual a capital da Estônia” não precisam de pensamento prolongado, e a tentativa do reasoning model de raciocinar sobre o problema apenas adiciona oportunidade de divergir. Saber identificar essa categoria é parte do que separa o operador maduro do iniciante deslumbrado.

Diagrama 14B.1 — Pirâmide de Complexidade × Custo de Raciocínio

Modo de raciocínio	Mecânica	Custo relativo	Latência relativa	Quando usar
Reasoning model dedicado (DeepSeek R1, Claude reasoning, OpenAI família o)	Cadeia interna de 10k-100k+ tokens, modelo treinado para raciocinar	10x-50x	10x-30x	Tarefa crítica com cadeia longa de dedução, auditoria do raciocínio importa, latência alta é tolerável
Few-shot CoT em modelo padrão	Exemplos completos no prompt, raciocínio guiado por imitação visível	2x-5x	2x-5x	Domínio com padrão repetível, exemplos próximos da tarefa são conhecidos
Zero-shot CoT em modelo padrão	Instrução “pense passo a passo”, raciocínio espontâneo e visível	1,5x-2x	1,5x-2x	Tarefa que beneficia de cadeia curta, exemplo seria custoso de produzir
Resposta direta em modelo padrão	Sem instrução de raciocínio, máxima velocidade	baseline	baseline	Tarefa simples, alta tolerância a erro, volume crítico

Conforme se desce a tabela, o custo, a latência e (quando aplicável) a qualidade caem. Conforme se sobe, a profundidade do raciocínio cresce — e o orçamento também.

□ **Diagrama 14B.1** — A escolha não é “qual o melhor modo de raciocínio”, é “qual o modo adequado para esta tarefa, com este custo, com esta latência, com este nível de auditabilidade exigido”. Subir na pirâmide só compensa quando o ganho de qualidade é mensurável e a tolerância a custo e latência é compatível.

14B.4 — EXEMPLO MEMORÁVEL: O ESCRITÓRIO DE M&A QUE MIGROU PARA REASONING

Cenário ilustrativo — composto a partir de padrões observados em escritórios de advocacia M&A de médio porte no Brasil entre 2024 e 2026, sem identificação de cliente específico.

Um escritório de advocacia brasileiro de médio porte, especializado em fusões e aquisições para empresas de capital fechado com tickets entre cinquenta milhões e oitocentos milhões de reais, com cerca de trinta sócios e duzentos advogados associados, sediado em São Paulo com filial em Belo Horizonte, vinha desde meados de 2024 usando um sistema interno baseado em modelo de propósito geral grande, com prompt cuidadosamente desenhado em few-shot Chain of Thought, para apoiar advogados associados em análise de risco em due diligence jurídica. O sistema lia documentos societários, contratos materiais, certidões e laudos, e produzia para cada operação um relatório estruturado de risco com hipóteses críticas identificadas, alertas em tópicos sensíveis como passivo trabalhista, contingências fiscais, e cláusulas que tipicamente comprometem closing.

O sistema funcionava bem em operações de complexidade média, com taxa de concordância com o sócio sênior revisor de cerca de oitenta e quatro por cento em revisões cruzadas, e tinha reduzido em

torno de quarenta por cento o tempo médio de produção do parecer inicial de associado antes da revisão do sócio. O problema era reincidente, e foi mapeado pelo head de tecnologia jurídica do escritório, profissional com background híbrido em direito e ciência da computação, ao longo do primeiro trimestre de 2025. Em operações complexas, especialmente as que envolviam estruturas societárias com múltiplos níveis de holdings, contratos de longo prazo com cláusulas interdependentes, e contingências fiscais que dependiam de interpretação cruzada de várias legislações, o sistema falhava em pontos sutis com consequência alta. Em uma operação específica, deixou de identificar um risco trabalhista derivado de cláusula em acordo coletivo regional que interagira de forma não óbvia com a estrutura de transferência de empregados pretendida na operação, e o sócio responsável só descobriu o ponto na revisão final, com custo de retrabalho que afetou o prazo de signing.

A equipe técnica fez uma análise estruturada das falhas, examinando trinta operações em que o sistema tinha errado em pontos identificáveis, e descobriu um padrão comum. Em todos os casos, o erro não era ignorância factual nem alucinação no sentido tradicional. O sistema tinha lido os documentos corretos, tinha identificado os elementos individuais relevantes, mas tinha falhado em executar a cadeia longa de raciocínio que conectava esses elementos através de três a sete passos lógicos interdependentes para chegar à conclusão de risco. Era exatamente o perfil de tarefa em que a literatura técnica indicava ganho significativo de reasoning models sobre Chain of Thought em modelo de propósito geral, e o head de tecnologia jurídica propôs um piloto controlado.

O piloto envolveu migrar a etapa de análise de risco em operações classificadas como complexas para um reasoning model dedicado, mantendo o modelo de propósito geral para a etapa de extração e estruturação inicial, e mantendo a revisão final humana inalterada. O reasoning model recebia o conjunto estruturado pré-processado pelo modelo padrão, junto com instruções detalhadas sobre como articular o raciocínio em torno de hipóteses de risco com cadeia de dedução explícita. Em paralelo, o time implementou três medidas de instrumentação que vinham diretamente das disciplinas que o Capítulo 14 cobre. Primeiro, dashboard de custo por operação segmentado por funcionalidade, para acompanhar o impacto financeiro da migração. Segundo, eval estruturado em que sócios sênior revisavam às cegas as análises produzidas pelos dois sistemas em uma amostra paralela, com identidade do gerador oculta, atribuindo nota e identi-

ficando erros. Terceiro, instrumentação de latência por etapa, para verificar se a maior demora do reasoning model criava gargalo no fluxo geral.

Os resultados ao longo de quatro meses, em uma amostra de cento e doze operações complexas, foram significativos em três dimensões mas com nuances que vale registrar. A taxa de concordância com sócios sêniores em análise de risco subiu de oitenta e quatro por cento para noventa e três por cento, com a maior parte do ganho concentrada exatamente na categoria de erros de cadeia longa de raciocínio que motivou o piloto. O custo por operação na etapa de análise de risco aumentou cerca de oito vezes em relação ao baseline, dentro da faixa que a equipe tinha modelado como aceitável para o segmento, e o ROI foi positivo porque o tempo evitado de retrabalho do sócio sênior em casos de erro era materialmente superior ao custo adicional de tokens. A latência por análise passou de cerca de quarenta segundos para cerca de três a cinco minutos, o que foi acomodado mudando o fluxo do produto para entregar análise como artefato assíncrono em vez de resposta em tempo real, decisão que exigiu trabalho conjunto entre tecnologia e operações jurídicas.

Houve ainda um aprendizado que a equipe não tinha antecipado, e que vale registrar como lição estrutural. A cadeia de raciocínio gerada pelo reasoning model, ainda que de qualidade superior, criou um problema de auditabilidade que o sistema anterior não tinha. Os sócios sêniores, ao revisarem análises, passaram a se apoiar fortemente no raciocínio articulado pelo modelo como evidência de robustez da conclusão, sem necessariamente questionar se aquele raciocínio refletia o cálculo real do modelo, conforme a literatura sobre faithfulness vinha alertando. Em três operações ao longo do piloto, sócios identificaram inconsistências em que a conclusão final era razoável mas o raciocínio articulado tinha lacunas ou pequenas contradições lógicas que sugeriam que o modelo tinha chegado à resposta certa por caminho diferente do exposto. A equipe respondeu instituindo um protocolo formal em que análises de operações classificadas como críticas precisam ter o raciocínio articulado revisado independentemente da conclusão, tratando ambos como artefatos de qualidade que precisam ser auditados em separado.

PARA EXECUTIVOS

Reasoning models entregam ganho mensurável em domínios com cadeia longa de dedução, e em jurídico de M&A esse ganho costuma justificar o custo adicional, mas a decisão de migrar um fluxo exige instrumentação prévia de custo, qualidade e latência, e exige protocolo explícito sobre como tratar o raciocínio articulado pelo modelo. Tratar o raciocínio exposto como evidência incondicional de qualidade do processo decisório é forma sofisticada de errar, e em domínios regulados ou com alta consequência de erro essa armadilha precisa ser endereçada no design da aplicação, não descoberta em incidente.

14B.5 — QUANDO USAR E QUANDO EVITAR

Sinal	Use reasoning model	Use CoT em modelo padrão	Use resposta direta
Tarefa exige cadeia longa de dedução interdependente	sim	parcial	não
Domínio tem verificador objetivo da resposta final (matemática, código)	sim	sim	não
Latência aceita acima de dez segundos por chamada	sim	sim	sim

Sinal	Use reasoning model	Use CoT em modelo padrão	Use resposta direta
Custo por chamada precisa ficar próximo de modelo padrão	não	parcial	sim
Fluxo conversacional em tempo real	não	não	sim
Tarefa rotineira com baixo custo de erro	não	não	sim
Tarefa criativa livre sem resposta certa	não	parcial	sim
Necessidade de auditabilidade total da cadeia	parcial	sim	não se aplica
Decisão regulada com alta consequência de erro	sim com protocolo	sim	não
Volume muito alto e margem por chamada estreita	não	parcial	sim
Conhecimento factual direto, não raciocínio	não	não	sim

A leitura sintética da tabela é que reasoning model é categoria adicional, com perfil próprio, que ganha em casos identificáveis e perde em outros igualmente identificáveis. Operações maduras desenvolvem heurísticas explícitas de roteamento, e tratam a escolha como decisão arquitetural que precisa ser instrumentada e validada com dados, não como preferência subjetiva do engenheiro.

14B.6 — VANTAGENS E LIMITAÇÕES

Dimensão	Vantagem	Limitação
Qualidade em raciocínio profundo	Mensuravelmente superior em cadeia longa de dedução, especialmente em matemática, código complexo, planejamento multipasso	Ganho não se materializa em tarefas onde raciocínio profundo não é o fator limitante
Auditabilidade	Cadeia articulada apoia revisão humana, dá pista de erro quando há, ajuda a justificar decisão em domínio regulado	Faithfulness não garantida — raciocínio exposto pode ser racionalização pós-hoc, não cálculo real
Custo por chamada	Justificável em decisões com alto custo de erro humano	Cinco a vinte vezes mais tokens por chamada, tier comercial separado tipicamente mais caro
Latência	Aceita em fluxos assíncronos, análises de fundo, processos batch	Inviabiliza fluxos conversacionais e qualquer caso com tolerância de poucos segundos
Robustez ao prompt	Menos sensível a variações superficiais de prompt do que CoT instruído	Mais sensível a configuração de profundidade de pensamento e parâmetros do reasoning, que precisam ser calibrados

Dimensão	Vantagem	Limitação
Ganho marginal de capacidade	Resolve classes de problema que modelos padrão não resolvem mesmo com CoT cuidadoso	Em muitas tarefas práticas o ganho marginal é zero, e o custo adicional é desperdício
Composição com outras camadas	Combina bem com RAG quando o conhecimento factual é estruturado e o raciocínio é o gargalo	Combina mal com fluxos de agentes onde múltiplas chamadas seriam necessárias, porque a latência composta vira inviável

14B.7 — CONEXÕES COM OUTROS CAPÍTULOS

- **Chain of Thought e raciocínio em modelo de propósito geral** → Capítulo 10. Este capítulo é leitura obrigatória anterior, porque reasoning models são compreendidos a partir do contraste com as três formas de invocar raciocínio descritas lá.
- **Context engineering em produção com cadeias longas** → Capítulo 11. A fase de pensamento de reasoning models compete por contexto e exige disciplina específica de gestão de janelas.
- **AI Engineering e a stack moderna em sete camadas** → Capítulo 14. Reasoning models adicionam complexidade nas camadas de observabilidade, custo, e roteamento entre modelos, e precisam ser tratados como categoria explícita na stack.
- **Comparação dos principais modelos do mercado** → Capítulo 15. A presença ou ausência de modo de raciocínio dedicado, e a forma como cada provedor expõe a cadeia, é critério estrutural de comparação que precisa entrar na matriz de escolha.
- **Economia de tokens em produção** → Capítulo 18. Tokens de pensamento entram na fatura e compõem o Custo Composto (Princípio 5), exigindo instrumentação e roteamento conforme detalhado em 14B.3.4.
- **Framework F2 — ENCAIXE-5 e a matriz de escolha de modelo** → adiciona o eixo “profundidade de raciocínio exigida” às

dimensões já cobertas, refinando a decisão arquitetural de qual modelo usar para cada caso de uso.

- **Framework F7 — Custo Composto** → o tier de reasoning é categoria adicional separada dos tiers padrão, com perfil próprio de cobrança que precisa ser modelado explicitamente em qualquer projeção financeira de aplicação que use reasoning models.
- **Framework F8 — Pirâmide da Avaliação** → reasoning models exigem nova camada de eval, especificamente voltada para testar faithfulness do raciocínio articulado, e essa camada precisa ser instituída explicitamente quando a aplicação for usada em domínio regulado.

14B.8 — RESUMO EXECUTIVO

Conceito	Síntese
Reasoning model	Classe de LLM treinada para reservar fase de pensamento prolongada antes da resposta, com cadeia interna que pode ser parcialmente oculta e que consome ordem de magnitude mais tokens que CoT instruído
Blueprint DeepSeek R1	Demonstrou que aprendizado por reforço puro, sem fase supervisionada inicial, produz comportamento de raciocínio emergente quando há verificador objetivo da resposta final (Nature 2025)
Diferença de CoT	Não é instrução de prompt, é arquitetura treinada; cadeia muito mais longa, parcialmente oculta, com perfil próprio de custo e latência
Faithfulness	Cadeia articulada não garante reflexo do cálculo interno; em casos identificáveis é racionalização pós-hoc, exigindo cautela em domínios regulados (Lanham et al., Anthropic)

Conceito	Síntese
Custo	Tokens de pensamento entram na fatura mesmo quando ocultos; tier comercial separado tipicamente mais caro; multiplicação de cinco a vinte vezes por chamada
Latência	Dezenas de segundos a alguns minutos por chamada, inviabilizando fluxos conversacionais e exigindo arquitetura assíncrona
Onde ganha	Cadeia longa de dedução, matemática avançada, código complexo, planejamento multipasso, decisão regulada com cadeia de raciocínio crítica
Onde perde	Tarefa rotineira, fluxo conversacional, conhecimento factual direto, criatividade livre, volume alto com margem estreita
Disciplinas operacionais	Roteamento inteligente entre tiers, instrumentação granular de custo, eval específico de faithfulness, protocolo de auditoria do raciocínio em domínios sensíveis

14B.9 – CHECKLIST DO CAPÍTULO

- ☐ Distinguir, com precisão técnica, zero-shot CoT, few-shot CoT e reasoning model treinado
- ☐ Explicar por que o blueprint do DeepSeek R1 mudou a tração da pesquisa em reasoning models, sem entrar em versão ou número específico
- ☐ Articular a questão de faithfulness do raciocínio e suas implicações para domínios regulados
- ☐ Estimar a multiplicação de custo por chamada esperada ao migrar de CoT em modelo padrão para reasoning model dedicado

- ☐ Identificar três classes de tarefa onde reasoning model ganha de forma clara sobre CoT
 - ☐ Identificar três classes de tarefa onde reasoning model perde ou empata com CoT
 - ☐ Mapear, na própria operação, qual fluxo seria candidato a piloto de reasoning model e qual definitivamente não seria
 - ☐ Esboçar arquitetura de roteamento entre tiers, com critérios explícitos para encaminhar tarefa a modelo padrão ou reasoning
 - ☐ Listar as três disciplinas operacionais para controle de custo em reasoning models
 - ☐ Defender, em comitê, por que reasoning model não é simplesmente “o modelo melhor” e por que adotá-lo de forma indiscriminada é forma cara de errar
 - ☐ Reconhecer como reasoning models afetam os Frameworks F2, F7 e F8 e o que muda em cada um
 - ☐ Instituir protocolo de auditoria do raciocínio articulado em qualquer aplicação que use reasoning model em domínio sensível
-

14B.10 — PERGUNTAS DE REVISÃO

1. Por que o trabalho do DeepSeek R1 mudou a conversa em torno de reasoning models, e o que significa, em termos estruturais, que o comportamento de raciocínio prolongado emergiu de aprendizado por reforço puro sem fase supervisionada inicial?
2. Qual a diferença prática, em custo, latência e auditabilidade, entre zero-shot Chain of Thought em modelo de propósito geral e o uso de um reasoning model dedicado para a mesma tarefa?
3. O que a literatura sobre faithfulness sugere sobre a relação entre a cadeia de raciocínio articulada e o cálculo interno do modelo, e quais implicações isso tem para uso em domínios regulados?
4. Por que tokens de pensamento entram na fatura mesmo quando o provedor não expõe o conteúdo da cadeia ao usuário, e que disciplinas operacionais o operador maduro institui para controlar essa fonte de custo?
5. Em que tipo de tarefa reasoning model perde para modelo de propósito geral em modo zero-shot, e por que essa categoria é contraintuitiva para quem trata reasoning model como “o modelo melhor”?

6. Como reasoning models afetam o Framework F2 — ENCAIXE-5 e o que muda na matriz de escolha de modelo com a adição do eixo “profundidade de raciocínio exigida”?
 7. Em uma aplicação que sua organização opera hoje, qual protocolo você instituiria para auditar a fidelidade do raciocínio articulado pelo modelo, em vez de aceitar a cadeia como evidência incondicional de qualidade do processo decisório?
-

14B.11 — EXERCÍCIOS PRÁTICOS

Exercício 1 — Mapeamento de candidatos

Liste todos os fluxos de IA em produção na sua organização. Para cada fluxo, classifique em três categorias: definitivamente candidato a reasoning model, possivelmente candidato sob piloto, definitivamente não candidato. Justifique cada classificação em duas a três frases, ancorando nos critérios da seção 14B.5.

Exercício 2 — Piloto controlado

Escolha um fluxo classificado como candidato. Desenhe um piloto de quatro semanas comparando, em paralelo cego, o sistema atual e a versão com reasoning model. Defina antecipadamente as três métricas que vão fundamentar a decisão: qualidade contra revisor sênior, custo por chamada, latência. Estabeleça critério explícito de sucesso antes de rodar.

Exercício 3 — Auditoria de faithfulness

Em uma amostra de vinte chamadas a reasoning model do piloto, peça a um revisor humano com domínio que avalie a cadeia de raciocínio articulada como artefato independente da conclusão. A cadeia é coerente? Há lacunas lógicas? Há indícios de racionalização pós-hoc? Documente os achados e proponha protocolo de uso.

Exercício 4 — Roteamento de tiers

Esboce, para uma aplicação real, uma lógica explícita de roteamento entre resposta direta, CoT em modelo padrão, e reasoning model dedicado, com base em sinais detectáveis no input. Quais sinais classificam a tarefa em cada tier? Como o sistema falha graciosamente quando o sinal é ambíguo?

14B.12 — PROJETO DO CAPÍTULO

Construa um piloto de reasoning model em uma aplicação real da sua organização, com instrumentação completa de custo, qualidade e auditabilidade.

Escolha um fluxo em que o ganho de raciocínio profundo é plausível, idealmente envolvendo cadeia longa de dedução em domínio com consequência operacional relevante. Construa em paralelo, durante pelo menos seis semanas, a operação atual e a versão com reasoning model, com cinquenta a duzentos casos representativos passando pelos dois sistemas com identidade do gerador oculta para o revisor. Instrumente custo por chamada segmentado, latência por etapa, taxa de concordância com revisor sênior, e crie eval específico de faithfulness com amostra revisada manualmente. Ao fim do período, produza relatório de decisão articulando, com dados, se a migração se justifica, em que volume, com qual protocolo de uso, e quais salvaguardas devem ser instituídas em domínio sensível. Esse projeto é provavelmente o que vai te convencer de que reasoning model não é categoria mágica nem panaceia, é ferramenta com perfil próprio que merece a mesma seriedade arquitetural que qualquer outra escolha de stack.

14B.13 — REFERÊNCIAS PRINCIPAIS

◆ Papers fundamentais

- DeepSeek-AI. “*DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*”. Nature, 2025. — blueprint público completo do treinamento de reasoning model por RL puro.
- Lanham, T. et al. “*Measuring Faithfulness in Chain-of-Thought Reasoning*”. Anthropic, 2023. — experimentos sistemáticos sobre fidelidade do raciocínio articulado em modelos modernos.
- Anthropic. “*Tracing the Thoughts of a Large Language Model*”. 2024-2025. — série de trabalhos sobre interpretabilidade e relação entre processo interno e cadeia exposta.
- Wei et al. “*Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*”. 2022. — referência fundadora para contraste com reasoning models nativos.

- Kojima et al. “*Large Language Models are Zero-Shot Reasoners*”. 2022. — base para entender por que zero-shot CoT funciona em modelos modernos.

◆ **Recursos e cobertura técnica**

- **Anthropic — Extended thinking** — documentação do modo de raciocínio estendido em Claude.
- **OpenAI — Reasoning models** — documentação da família o.
- **DeepSeek — papers e código** — repositório oficial para acompanhamento de iterações pós-R1.
- Cobertura de Latent Space e Interconnects sobre reasoning models, com discussão técnica acessível e atualizada.

◆ **Apêndice Vivo**

- Apêndice J (Trilha do Número) — Apêndice Vivo, seção Reasoning Models, com versões correntes, faixas de preço por tier, e benchmarks atualizados. Tudo o que envolve número específico de versão, custo unitário ou benchmark mora lá, não neste capítulo.

14B.14 — Autoavaliação

#	Critério	Você consegue?
1	Clareza — Explicar para um CTO o que é reasoning model e em que difere de Chain of Thought instruído via prompt, em três minutos, usando a analogia do engenheiro que rascunha antes de comitar	☐

#	Critério	Você consegue?
2	Profundidade — Defender, em discussão técnica, por que o blueprint do DeepSeek R1 é estruturalmente importante, articulando RL puro como caminho de raciocínio emergente e a questão de faithfulness como ponto de cautela	☐
3	Aplicação — Mapear, na sua organização, pelo menos três fluxos onde reasoning model é candidato plausível, três onde definitivamente não é, e desenhar um piloto controlado para um deles	☐
4	Conexão — Articular como reasoning models afetam o Capítulo 10 (CoT), o Capítulo 14 (AI Engineering), o Capítulo 18 (custo), e os Frameworks F2, F7 e F8	☐
5	Curiosidade ativa — Está com vontade de instituir protocolo de auditoria de faithfulness em alguma aplicação atual, ou de comparar provedores de reasoning na próxima rodada de avaliação de stack	☐

□ **Você acabou de fechar a Parte 3 — Agentes e IA Moderna.**

“Modelos que pensam por mais tempo erram menos em problemas onde pensar resolve. Em problemas onde pensar não é o gargalo, pensar mais apenas faz a fatura crescer e a latência morder. O operador maduro sabe distinguir os dois casos antes de assinar o contrato.”

15. Comparação dos Principais Modelos

“No platô da fronteira, a pergunta não é mais qual modelo é o melhor. É qual modelo é o certo para esta tarefa específica, com este orçamento, com estas restrições.”

15.2 — ANALOGIA: A FROTA DE VEÍCULOS DA EMPRESA

Pense em como uma empresa logística pensa sobre sua frota de veículos. Ela não compra apenas um tipo de caminhão e usa para tudo. Para entregas urbanas pequenas, usa vans leves. Para cargas pesadas em longas distâncias, usa carretas. Para entregas em última milha em vielas estreitas, usa motos ou triciclos. Para cargas refrigeradas, usa frota especializada. Cada veículo é otimizado para um tipo de tarefa, e a operação madura sabe roteirizar a carga certa para o veículo certo, em tempo real.

Modelos de IA seguem lógica similar. Existe um modelo grande e premium ideal para tarefas que exigem raciocínio profundo, análise crítica ou escrita executiva, em que o custo por chamada é justificado pela qualidade da saída. Existe um modelo balanceado ideal para o grosso da operação, em que custo e qualidade precisam se equilibrar. Existe um modelo pequeno e barato ideal para tarefas simples em altíssimo volume, em que economia escala dramaticamente. Existe um modelo multimodal nativo ideal para vídeo e imagem. Existe um modelo open source ideal para dados sensíveis que não podem sair da infraestrutura. Cada um tem seu lugar, e a maturidade está em saber rotear, não em escolher um único.

A mentalidade que produz desperdício é tratar o modelo como commodity, comprando um único e usando indiscriminadamente. A mentalidade que produz vantagem é tratar o modelo como ferramenta especializada, com frota gerenciada em volta de critérios técnicos e econômicos claros.

| 15.3 — EXPLICAÇÃO TÉCNICA

15.3.1 — O panorama por padrão de especialização

A **Anthropic** opera a família Claude em três níveis canônicos: Opus (raciocínio premium, código de longa duração, escrita executiva), Sonnet (equilíbrio entre qualidade e custo para o grosso da produção corporativa) e Haiku (classificação e extração em volume, latência baixa). A força histórica relativa da família está em código em ambiente real, escrita avaliada por humanos em estudos cegos, e filosofia pública de alignment (Constitutional AI). É também a origem do padrão MCP, que se consolidou como referência aberta da indústria.

A **OpenAI** opera a família GPT em camadas premium e variantes reduzidas (mini, nano). A força relativa tende a aparecer em raciocínio matemático competitivo e em *computer use* (capacidade do modelo de operar interfaces gráficas como um humano faria, clicando, digitando, navegando), o que faz dela escolha frequente para agentes autônomos que manipulam software de terceiros.

A **Google DeepMind** opera o Gemini com modelo premium e variante de produção em volume (Flash). A força relativa é decisiva em multimodal — vídeo, imagem, áudio nativos — e em raciocínio abstrato sobre padrões visuais. Costuma também oferecer a maior janela de contexto entre os frontier proprietários e tende a posicionar-se em custo-benefício agressivo no nível premium.

A **xAI** opera o Grok, com acesso nativo em tempo real ao X e tolerância maior a temas sensíveis em comparação às três grandes. Tem nicho específico em monitoramento de redes sociais, jornalismo e análise de tendências em tempo real.

A **DeepSeek**, laboratório chinês, opera modelos para uso geral e variantes com thinking visível para raciocínio. Sua marca relativa é a relação custo-benefício extrema combinada à oferta open weights, que pode ser baixada e executada em infraestrutura própria. Essa combinação de custo e abertura tem reorganizado a discussão de viabilidade econômica em muitos casos de uso.

A **Z.AI** (modelos GLM) posiciona-se como alternativa chinesa open weights séria em benchmarks. A **Mistral** francesa segue relevante em modelos abertos com foco em eficiência. A **Meta** mantém a família Llama como referência principal em open source ocidental.

□ **Diagrama 15.1 — Pontos Fortes dos Principais Modelos em 2026**

PONTOS FORTES DOS PRINCIPAIS MODELOS — MAIO 2026

No platô da fronteira, capacidade bruta é parecida. As diferenças aparecem em especialização.

MODELO	LIDERANÇA	CUSTO (input/M)	QUANDO ESCOLHER
Claude Opus 4.6/4.7 Anthropic - frontier	Código (SWE-bench 80.8%), escrita raciocínio profundo	US\$ 15 premium	desenvolvimento, análise crítica conteúdo executivo, agentes
Claude Sonnet 4.5/4.6 Anthropic - balanceado	Custo-benefício em tarefas complexas o "cavalo de batalha" corporativo	US\$ 3 médio	aplicações em produção em volume RAG, agentes, automação
GPT-5.4 / 5.5 OpenAI - frontier	Matemática (AIME 2026 perfeito) computer use (OSWorld 75%)	US\$ 12 premium	workflows agentes autônomos raciocínio estruturado, plugins
Gemini 3.1 Pro Google - frontier	Multimodal (Video-MME 78.2%) ARC-AGI-2, contexto 2M tokens	US\$ 2 melhor custo	vídeo, imagem, documentos longos workloads Google Workspace
DeepSeek V3 / R1 DeepSeek - open weights	Raciocínio econômico, código R1 com thinking visível	US\$ 0,27 disruptivo	volume massivo, privacidade deploy próprio, exploração
Grok 4 xAI - frontier	Acesso ao X em tempo real tolerância maior a temas sensíveis	US\$ 5 médio	monitoramento de redes, jornalismo tendências sociais em tempo real
Llama 4 (open) Meta - open weights	Ecosistema open source dominante flexibilidade total	grátis (modelo) + infra	deploy on-premises, fine-tuning dados sensíveis, customização

Pontos fortes dos modelos

No platô da fronteira, capacidade bruta é parecida. As diferenças aparecem em especialização.

15.3.2 — As três grandes dimensões de comparação

Para comparar modelos com critério, vale separar três dimensões que costumam ser confundidas em discussões superficiais.

A primeira é **capacidade**, que é o quanto o modelo consegue executar tarefas complexas com qualidade. No regime de platô, os três frontier proprietários (Claude Opus, GPT premium, Gemini Pro) operam em faixa próxima, com cada um exibindo força relativa em um eixo: código e escrita executiva tendem a favorecer Claude, raciocínio matemático e computer use tendem a favorecer GPT, multimodalidade pesada tende a favorecer Gemini. Modelos open source costumam ficar à distância de uma ou duas gerações em benchmarks gerais, com variação por domínio. Para casos que exigem raciocínio na fronteira, os frontier proprietários ainda valem o preço; para casos

mais comuns, a diferença começa a não justificar custo. Os números específicos de cada rodada envelhecem rápido e devem ser consultados nas fichas técnicas atuais dos fornecedores.

A segunda é **economia**, que envolve custo por token, latência média, e escalabilidade de quota. O frontier premium costuma operar em faixa significativamente mais cara que os modelos balanceados, e estes, por sua vez, em faixa significativamente mais cara que os modelos pequenos e os open source rodando em infraestrutura própria. A regra de bolso, em vez de decorar preço por milhão (que muda em meses), é desenhar o stack assumindo três tiers e rotear cada chamada ao mais barato suficiente, conforme o **Framework Custo Composto em Três Tempos** (Cap 18).

A terceira é **alinhamento e segurança**, que cobre como o modelo responde a pedidos problemáticos, o quanto ele é manipulável via prompt injection, e o cuidado do laboratório com riscos sistêmicos. Anthropic se posiciona explicitamente como líder em alinhamento via Constitutional AI, com filosofia pública e investimento massivo em interpretabilidade. OpenAI tem práticas de segurança maduras com posicionamento público menos centrado em segurança. Google segue padrões corporativos sólidos. Modelos open source variam enormemente, com alguns tendo menos restrição que os frontier proprietários, o que pode ser vantagem ou desvantagem dependendo do caso. Para domínio sensível (saúde, jurídico, financeiro, educação), a filosofia de alignment do vendor pesa mais do que o benchmark do mês, conforme se aprofunda no Capítulo 23 — Alignment.

15.3.3 — Os benchmarks que ainda importam, lidos como padrão

A era em que um único número resumia capacidade do modelo já passou. MMLU, que era a referência em 2023, saturou perto de 90% e perdeu poder discriminatório. O benchmark sucessor seguirá o mesmo destino. O exercício útil aqui é entender **o que cada benchmark mede**, e usar esse mapa para julgar o que importa para o seu caso. A liderança em cada rodada deve ser consultada nas tabelas de benchmark mais recentes.

GPQA Diamond — raciocínio em ciências (física, química, biologia) com perguntas de nível doutorado. Mede a fronteira de raciocínio científico.

SWE-bench Verified e SWE-bench Pro — capacidade de resolver issues reais de GitHub. Verified usa problemas mais curados; Pro adiciona complexidade. O benchmark mais próximo do trabalho real de engenharia de software.

AIME — matemática competitiva avançada. Mede raciocínio formal sob restrição de tempo.

ARC-AGI-2 — raciocínio abstrato sobre padrões visuais. É um dos benchmarks que se mantém difícil por escapar de memorização.

OSWorld — *computer use*: capacidade do modelo de operar um sistema operacional como humano faria, clicando, digitando, navegando. Mede agência aplicada a software de terceiros.

Video-MME — compreensão de vídeo. Mede multimodalidade temporal.

Humanity's Last Exam — atualmente o benchmark mais difícil disponível, com perguntas que exigem expertise de fronteira em campos específicos. Nenhum modelo atinge metade da pontuação no momento desta edição.

A lição prática é que olhar para um único número é miopia. Olhar para a combinação de benchmarks relevantes ao seu caso de uso é o caminho. Modelos diferentes brilham em benchmarks diferentes, e essa heterogeneidade é o que justifica roteamento — princípio operacional do **Framework Diagnóstico de Encaixe entre Tarefa e Modelo** (Princípio 4).

15.4 — FRAMEWORK DE DECISÃO — Diagnóstico de Encaixe entre Tarefa e Modelo

O framework opera em árvore de perguntas, descendo conforme critérios concretos. Em cada nó, a resposta é sobre **padrão de tarefa**, não sobre o lançamento da semana.

Pergunta 1 — Dados sensíveis exigem rodar em sua infraestrutura? Se sim, modelos open source são o caminho, com Llama, Mistral, Qwen, DeepSeek e GLM como candidatos principais. Discussão de qual entre eles vai para o Capítulo 16. A liderança comparativa do momento entre eles muda a cada release — consulte o Apêndice J (a Trilha do Número).

Pergunta 2 — Custo é restrição dura, com volume alto e margem apertada? Se sim, vale considerar os modelos de menor custo dos frontier proprietários, sendo as variantes Flash, Haiku e Mini os candidatos óbvios. A escolha entre eles depende do eixo dominante (latência, qualidade em volume, custo-benefício multimodal); o ranking corrente está no Apêndice J.

Pergunta 3 — A tarefa envolve vídeo, imagem ou áudio em volume significativo? Se sim, o frontier multimodal do momento é a escolha mais clara — historicamente, a família Gemini lidera essa categoria com folga.

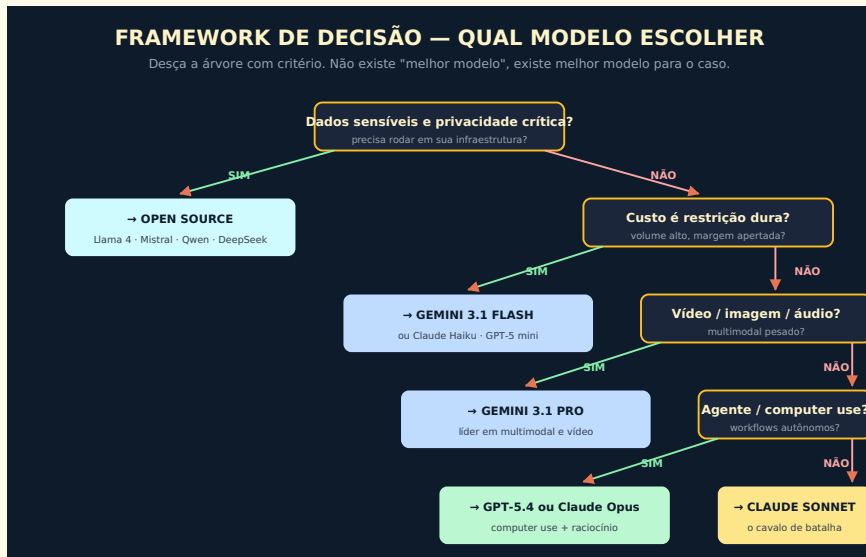
Pergunta 4 — O sistema vai operar como agente autônomo, com computer use ou workflows complexos de várias etapas? Se sim, há dois caminhos canônicos: o frontier com força em computer use (historicamente GPT premium), e a combinação Claude + Claude Code para tarefas centradas em software. A decisão entre eles passa pelo tipo de ferramenta que o agente vai operar.

Pergunta 5 — Nenhuma das anteriores cravou? O Sonnet (família balanceada do Claude) costuma ser o cavalo de batalha padrão para aplicações corporativas em produção, pela combinação de capacidade, custo, latência e ecossistema. Em outras famílias, a variante “balanceada” (não premium, não pequena) joga o mesmo papel.

Eixo	O que pontuar	Família que tende a vencer
Razão complexa	Tarefas que exigem raciocínio em várias etapas, com auditoria do percurso	Premium dos três frontier; thinking-models open
Código	Edição de código real, com testes e contexto longo	Claude Opus em SWE-bench Verified, historicamente
Contexto longo	Análise de documentos extensos ou bases de conhecimento volumosas	Gemini Pro pelo tamanho de janela típico
Multimodal	Vídeo, imagem, áudio nativos	Gemini Pro
Custo crítico	Volume altíssimo, margem apertada	Variantes Flash/Haiku/Mini; open source self-hosted

Esses padrões mudam pouco entre gerações. Os números concretos mudam toda rodada e devem ser consultados separadamente.

Diagrama 15.2 — Framework de Decisão



Framework de decisão

Desça a árvore com critério. Não existe “melhor modelo”, existe melhor modelo para o caso.

Esse framework tem uma virtude que vale destacar. Ele torna a decisão articulável e auditável, em vez de ser intuição mascarada de análise técnica. Quando alguém propõe migrar para o lançamento da semana, você consegue perguntar “passamos pelas perguntas anteriores?” e exigir justificativa de cada decisão. Isso muda a qualidade da conversa em organizações em que escolhas de modelo costumavam ser orientadas por preferência pessoal de quem propôs.

15.5 — EXEMPLO MEMORÁVEL: A EMPRESA QUE ECONOMIZOU US\$ 380 MIL COM ROTEAMENTO

△ **Cenário ilustrativo** — composto a partir de padrões observados em operações reais de fintech brasileira; números e nomes são realistas mas não identificam empresa específica.

Uma fintech brasileira de cartão de crédito operava toda sua infraestrutura de IA com o modelo premium do frontier proprietário disponível na época, escolhido inicialmente por entregar a melhor qualidade em testes iniciais. O sistema tinha cerca de meio milhão de chamadas mensais, cobrindo desde análise de risco de crédito até suporte ao cliente, passando por classificação automática de transações suspeitas e redação de comunicados regulatórios. A conta mensal estava em torno de US\$ 47 mil dólares, com tendência de crescimento conforme o produto escalava.

Uma consultoria foi contratada para auditar a operação de IA e identificar oportunidades de economia. A descoberta principal não foi sobre prompts ou caching, foi sobre roteamento. A empresa estava usando o modelo mais caro do mercado para tarefas que poderiam rodar em modelos significativamente mais baratos sem perda de qualidade.

A análise classificou as chamadas em quatro categorias de complexidade.

A **categoria 1**, com cerca de 60% do volume, era composta de tarefas simples estruturadas como classificação de transações (“essa compra é categoria alimentação ou transporte?”), extração de dados de mensagens curtas, validação de campos em formulários. Para essas, testes mostraram que o modelo pequeno da mesma família (Haiku) entregava qualidade praticamente idêntica ao premium, ao custo de fração baixa do preço.

A **categoria 2**, com cerca de 25% do volume, era composta de tarefas moderadas como redação de respostas padrão a clientes, sumarização de tickets, análise inicial de fraude. Para essas, o modelo balanceado da mesma família (Sonnet) entregava qualidade equivalente ao premium em testes cegos com avaliadores humanos, ao custo de cerca de um quinto.

A **categoria 3**, com cerca de 12% do volume, era composta de tarefas complexas como análise de risco crítico, redação de comunicados regulatórios para o Banco Central, e investigação aprofundada de fraude. Para essas, o premium continuava sendo a melhor escolha, e o custo se justificava pela qualidade.

A **categoria 4**, com cerca de 3% do volume, era composta de tarefas com vídeo ou imagem (verificação de documentos com foto). Para essas, migrar para o frontier multimodal (família Gemini Pro) entregou qualidade superior em multimodal a custo menor.

A implementação do roteamento levou cerca de três semanas. Construíram um classificador leve no início do pipeline, usando Claude Haiku, que olhava cada chamada e decidia para qual modelo encaminhar. O classificador em si rodava por uma fração de centavo, e o roteamento decidia onde gastar.

O resultado, três meses após estabilização, foi redução de US\$ 47 mil para cerca de US\$ 15,5 mil mensais, ou seja, economia de US\$ 380 mil anualizada, mantendo a qualidade percebida pelos usuários em todas as categorias. Mais importante, a empresa ganhou flexibilidade arquitetural, podendo trocar modelos em cada categoria conforme novas versões aparecem, sem refazer a aplicação inteira.

A lição estrutural não foi sobre escolher o modelo certo no momento zero, foi sobre arquitetura. **Sistemas maduros de IA tratam modelo como configuração, não como decisão de fundação. Quando você consegue trocar modelos por categoria sem reescrever a aplicação, ganha capacidade de otimizar continuamente conforme o mercado evolui.** Em uma indústria que lança modelos novos a cada três meses, essa flexibilidade vira vantagem competitiva real.

PARA EXECUTIVOS

Se sua organização usa um único modelo para tudo, é altamente provável que esteja pagando entre 3x e 8x mais do que precisaria para a qualidade que entrega. Implementar roteamento entre modelos por categoria de tarefa é um dos investimentos com maior ROI imediato em qualquer operação de IA em escala.

15.6 — TENDÊNCIAS QUE VALEM ACOMPANHAR

Vale terminar com três tendências em curso que vão afetar decisões de modelo nos próximos dois ou três anos.

A primeira é a **comoditização gradual da capacidade frontier**. Modelos open source estão fechando o gap com frontier proprietários em ritmo acelerado, e em 2026 já há paridade em muitas tarefas comuns. Em três anos, é provável que o que hoje é frontier seja commodity acessível em hardware modesto, e o frontier do momento seja qualitativamente diferente. Decisões arquiteturais que assumem dependência permanente de um provedor específico provavelmente vão envelhecer mal.

A segunda é a **proliferação de modelos especializados**. Em vez de um modelo gigante que faz tudo, está crescendo o número de modelos menores otimizados para domínios específicos como código, medicina, direito, biologia. Em muitos casos, esses modelos especializados superam frontier generalistas em seus domínios, ao custo de fração do preço. Organizações que operam em domínios específicos vão se beneficiar de avaliar essas alternativas verticais.

A terceira é a **integração com hardware específico**. NPUs em laptops, GPUs especializadas em data centers, e chips dedicados como Trainium da AWS, Tensor Processing Units do Google, e silicon proprietário da Apple, estão mudando a economia de inferência. Modelos que rodam bem em hardware específico podem ter custo radicalmente diferente em produção, e essa dimensão entrou definitivamente no critério de decisão.

15.7 — CONEXÕES COM OUTROS CAPÍTULOS

- **Como modelos funcionam por dentro:** Capítulo 2
- **Tokens e custo de operação:** Capítulo 3
- **Fine-tuning como alternativa a trocar de modelo:** Capítulo 8
- **AI Engineering e roteamento entre modelos:** Capítulo 14
- **Modelos open source em profundidade:** Capítulo 16

- **Todos os modelos Claude e como escolher:** no Livro 2
- **Economia de tokens em produção:** Capítulo 18

15.8 — RESUMO EXECUTIVO

Conceito	Síntese
Platô da fronteira	Capacidade bruta dos frontier converge; diferenças aparecem em especialização por eixo
Frontier proprietários	Família Claude, família GPT, família Gemini — líderes correntes no Apêndice J
Frontier open source	Família Llama, DeepSeek, Qwen, GLM, Mistral — comparativo no Apêndice J
Força relativa típica em código e escrita	Família Claude
Força relativa típica em matemática e computer use	Família GPT
Força relativa típica em multimodal e contexto longo	Família Gemini
Custo-benefício no frontier	Tipicamente capturado pelo Gemini premium, com flutuação por rodada — ver Apêndice J
Framework de decisão	Diagnóstico de Encaixe entre Tarefa e Modelo: sensibilidade → custo → multimodal → agência → default balanceado
Roteamento inteligente	Reduz custo de forma expressiva sem perda de qualidade; arquitetural, não de prompt
Onde estão os números	Versões, preços e benchmarks devem ser conferidos em fontes oficiais atualizadas

15.9 — CHECKLIST DO CAPÍTULO

- ☐ Citar os três frontier proprietários de 2026 e o que cada um lidera
 - ☐ Distinguir as três dimensões de comparação (capacidade, economia, alinhamento)
 - ☐ Reconhecer os seis benchmarks que ainda diferenciam frontier em 2026
 - ☐ Aplicar o framework de decisão para um caso real
 - ☐ Defender, em uma reunião, por que usar um único modelo para tudo é desperdício
 - ☐ Identificar três tendências em curso que vão afetar escolhas nos próximos anos
-

15.10 — PERGUNTAS DE REVISÃO

1. Por que MMLU perdeu poder discriminatório como benchmark, e que padrão se repete com benchmarks sucessores?
2. Em que situação um modelo Gemini Pro é a escolha mais clara, e por quê o eixo “multimodal” tende a manter essa força entre gerações?
3. Como você decidiria entre Claude Opus e o frontier GPT para um sistema de agentes autônomos? Quais perguntas do Diagnóstico de Encaixe entre Tarefa e Modelo você usaria?
4. Por que roteamento entre modelos costuma render mais que negociar desconto com um único provedor?
5. Que tendência você acompanha mais de perto, e como ela afeta decisões de longo prazo da sua organização?

15.11 — EXERCÍCIOS PRÁTICOS

Exercício 1 — Inventário e classificação

Liste as chamadas de IA da sua organização nos últimos 30 dias, agregadas por funcionalidade. Classifique cada uma em uma de quatro categorias de complexidade. Estime potencial de economia se aplicar roteamento.

Exercício 2 — Comparação de saída

Pegue cinco prompts representativos da sua operação. Rode cada um em três modelos diferentes de famílias distintas (por exemplo, um tier médio da família Claude, um tier econômico da família GPT e um Flash da família Gemini; versões pontuais no Apêndice J). Compare qualidade às cegas, sem saber qual produziu qual. Documente o resultado.

Exercício 3 — Análise de benchmark relevante

Identifique qual benchmark de 2026 melhor representa o tipo de tarefa que sua organização executa em IA. Pesquise como cada fronteira se sai nesse benchmark. Use isso como input para decisões de roteamento.

Exercício 4 — Esboço de roteador

Para uma aplicação real, esboce a lógica de um roteador. Que sinais usar para classificar a chamada? Que mapeamento de categoria para modelo aplicar? Estime ganho potencial.

15.12 — PROJETO DO CAPÍTULO

Implemente roteamento entre modelos em uma aplicação real.

Escolha a aplicação de IA com maior volume da sua organização. Implemente um roteador simples no início do pipeline, classificando cada chamada em três a quatro categorias de complexidade. Mapeie cada categoria para um modelo apropriado. Execute em paralelo com a configuração antiga por duas semanas, comparando custo, latência e qualidade. Documente o resultado. Esse projeto, mesmo em

escala modesta, costuma render entre 30% e 70% de economia mantendo qualidade, e prepara terreno conceitual para todas as decisões de modelo futuras.

15.13 — REFERÊNCIAS PRINCIPAIS

◆ Leaderboards e benchmarks

- [Vellum LLM Leaderboard](#)
- [Artificial Analysis](#)
- [LM Council Benchmarks](#)
- [LMSYS Chatbot Arena](#)

◆ Papers sobre benchmarks atuais

- *“GPQA: A Graduate-Level Google-Proof Q&A Benchmark”*. 2023.
- *“SWE-bench: Can Language Models Resolve Real-World GitHub Issues?”*. 2023.
- *“ARC-AGI-2: Abstract Reasoning Corpus, second generation”*. 2024.
- *“OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments”*. 2024.

◆ Documentação dos provedores

- [Anthropic Models](#)
- [OpenAI Models](#)
- [Google Gemini](#)
- [DeepSeek API](#)

15.14 — Autoavaliação

#	Critério	Você consegue?
1	Clareza — Explicar por que não existe “melhor modelo” para um executivo em 90 segundos, usando a analogia da frota	☐

#	Critério	Você consegue?
2	Profundidade — Defender, em discussão técnica, escolha de modelo para um caso real, citando benchmarks relevantes	☐
3	Aplicação — Aplicar o framework de decisão a três casos reais da sua organização	☐
4	Conexão — Articular como escolha de modelo se conecta com tokens (Cap 3), fine-tuning (Cap 8), AI Engineering (Cap 14), open source (Cap 16)	☐
5	Curiosidade — Está com vontade de entender em profundidade o ecossistema open source, suas vantagens reais e suas armadilhas	☐

“Em 2026, escolher modelo não é decisão de fundação, é decisão de roteamento. Quem entende isso opera com flexibilidade. Quem ignora, paga conta de quem opera.”